

CAMPUSCONNECT

Connect. Collaborate. Grow.

Consolidated Project Report

Software Engineering & Digital Marketing / E-Commerce Departments

University of Buea — Faculty of Engineering and Technology

August 2026

Source documents consolidated: UI/UX Design Documentation · Updated System Analysis (Arrey) · Database Design Documentation (Siela Lucien Noumboum) · Requirement Analysis (SRS/FRD/NFRD) · Digital Marketing & E-Commerce Full Project Report · Live project codebase

Table of Contents

Table of Contents	2
1. Project Overview	5
1.1 Document Scope	5
2. Software Requirements Specification	6
2.1 Overall Description.....	6
2.2 User Classes and Characteristics	6
2.3 System Interfaces	6
2.4 Functional Requirements (Summary).....	6
2.5 Non-Functional Requirements (Summary).....	7
Performance	7
Security & Data Integrity.....	7
Reliability & Availability	7
Usability & Accessibility	7
2.6 Selected User Stories.....	7
3. System Architecture & Modules	9
3.1 Architecture Overview	9
3.2 System Modules	9
3.3 Actors.....	10
3.4 Core Workflows	10
3.5 Class Diagram (Entities).....	10
3.6 Technical Feasibility	10
4. Database Design (Cloud Firestore)	11
4.1 Collection Structure.....	11
4.2 Key Collections	11
users.....	11
marketplace.....	12
chats.....	12
4.3 Design Rationale.....	12
4.4 Security Rules — Structural Sketch.....	12
5. UI/UX Design Documentation.....	14
5.1 User Flow	14
5.2 Information Architecture	14

5.3 Key Screens	14
6. Payment Gateway Addition	15
6.1 Chosen Provider	15
6.2 Flow.....	15
6.3 Schema Addition	15
7. Design Consistency & Reconciliation	16
7.1 Collection & Module Naming.....	16
7.2 Messaging Thread Terminology.....	16
7.3 Status Value Casing	16
7.4 Soft-Delete Implementation	16
7.5 Database Technology	16
7.6 Monetization vs. Payment Processing Scope	17
7.7 Outstanding Items	17
8. Digital Marketing & E-Commerce Report.....	18
8.1 Brand Identity.....	18
Logo.....	18
Color Palette	18
Typography & Tone	18
Tagline.....	18
8.2 Market Research	18
Target Audience	18
Competitive Landscape	18
User Personas	19
Market Opportunities	19
8.3 Marketing Strategy.....	19
Go-to-Market.....	19
Social Media	19
Launch Campaign	19
Growth & Acquisition.....	19
8.4 Content Creation	20
8.5 Business Strategy	20
Revenue Model	20
Pricing	20
Retention & Support	20
9. Status Summary & Next Steps	21

Next Steps..... 21

1. Project Overview

CampusConnect is a web-based platform designed specifically for university students. It allows students to buy and sell used items, advertise their skills and services, discover internship and job opportunities, and stay informed about campus events — all within one trusted, student-only platform.

The project is delivered as a cross-functional collaboration between two departments functioning as one team: the Software Engineering team, responsible for system analysis, architecture, database design, and implementation, and the Digital Marketing & E-Commerce team, responsible for brand identity, market research, marketing strategy, and go-to-market execution. This report consolidates the completed work of both teams into a single reference document, along with a reconciliation of naming and scope differences that emerged as the design evolved across documents.

1.1 Document Scope

This report draws together the following source documents, each originally authored separately:

- UI/UX Design Documentation — user flow, information architecture, navigation structure, and low-fidelity wireframes
- Updated System Analysis (prepared by Arrey) — system architecture, the 11 core system modules, workflow diagrams, and full UML documentation (use case, activity, sequence, and class diagrams)
- Database Design Documentation (prepared by Siela Lucien Noumboum, Team Member 5) — the Cloud Firestore schema, collection structure, and data dictionary
- Requirement Analysis — the Software Requirements Specification (SRS), Functional and Non-Functional Requirements Documents (FRD/NFRD), user stories, detailed use cases, project scope, and Requirement Traceability Matrix (RTM)
- Digital Marketing & E-Commerce Full Project Report — brand development, market research, marketing strategy, content creation, and business strategy
- The live project codebase — a Vite + Firebase JavaScript application, used as the final authority for naming and implementation questions where the design documents disagreed

2. Software Requirements Specification

2.1 Overall Description

CampusConnect is a modular web application structured around a clean, service-oriented architecture, serving student buyers, sellers, service providers, event organizers, and student job seekers.

2.2 User Classes and Characteristics

- Student User: browses, posts marketplace items, offers services, applies for jobs, RSVPs to events, and refers peers.
- Verified Recruiter / Alumni: posts employment opportunities and reviews submitted PDF resumes.
- Campus Administrator / SGA: publishes official campus announcements and monitors system health.

2.3 System Interfaces

- User Interface: web and mobile interfaces compliant with WCAG 2.1 Level AA standards.
- Firebase SDK Interface: direct client-side integration with Firebase Auth, Cloud Firestore, and Cloud Storage.
- Search Engine Interface: Cloud Firestore queries augmented with the Firebase Algolia Search Extension for full-text search.

2.4 Functional Requirements (Summary)

ID	Module	Requirement	Priority
FR-AUTH-01/02	Auth	Restrict registration to institutional email; send automated verification.	High
FR-MKT-01/02	Marketplace	Create/edit/soft-delete/mark-sold listings; up to 5 images per listing.	High/Med
FR-SRV-01	Services	Create service posts with hourly rates, skill tags, portfolio links.	High
FR-JOB-01/02	Opportunities	Post internships/jobs with deadlines; upload PDF resume and apply.	High
FR-EVT-01	Events	Schedule events with location, date, time, category, capacity.	Medium
FR-MSG-01	Messaging	Real-time messaging between buyers/clients and sellers/providers.	High
FR-SCH-01	Search/Filter	Global search and filter by category, price, condition, date.	High
FR-NOTIF-01	Notifications	In-app and email notifications for matches and platform updates.	Medium
FR-REF-01	Referral	Generate unique referral links/codes and track referrals.	Low

ID	Module	Requirement	Priority
FR-RATE-01	Trust & Safety	1–5 star ratings and written feedback after interactions.	High
FR-ANC-01	Announcements	Admins/ reps publish public announcements and system updates.	Medium
FR-PAY-01 (new)	Payments	Process boosted-listing and premium-subscription payments via MTN MoMo / Orange Money, mediated by Cloud Functions, with no payment credentials stored in Firestore.	High

2.5 Non-Functional Requirements (Summary)

Performance

- Initial page load must not exceed 2.0 seconds over standard 4G connectivity.
- Chat messages dispatched via Firestore onSnapshot listeners must render on receiving clients in under 500 milliseconds.

Security & Data Integrity

- Firestore Security Rules must enforce user-level data isolation (auth.uid must match the record's owner field).
- Deletions across marketplace items and service listings must use a soft-delete pattern rather than hard deletion, preserving records for moderation and dispute auditing.
- All media attachments undergo multi-layered verification: client-side magic-byte checks, Cloud Storage MIME-type/size rules, and asynchronous Cloud Function malware scanning before promotion to public storage.
- NFR-SEC-04 (new): payment credentials are never stored in Firestore or Cloud Storage — only a gateway reference (gatewayRef) is retained; provider API keys live in Cloud Functions environment configuration, never in client code.

Reliability & Availability

- 99.5% uptime target during active academic semesters; incremental backups every 24 hours with an RPO of 1 hour or less.

Usability & Accessibility

- WCAG 2.1 Level AA compliance; fully responsive layout from 320px to 2560px viewport widths.

2.6 Selected User Stories

- As a student buyer, I want to filter marketplace listings by category and price range so I can quickly find affordable textbooks.
- As a graduating student, I want to upload photos of my furniture so incoming students can buy my items before move-out.
- As a student tutor, I want to list my hourly rate and portfolio so peers can hire me for academic support.

- As a buyer, I want real-time direct messaging with a seller so we can arrange a safe campus pickup spot.
- As an active user, I want push notifications about new listings in my major so I can catch sales immediately.
- As a student user, I want to share a referral code with classmates to expand our campus peer network.

3. System Architecture & Modules

3.1 Architecture Overview

CampusConnect is delivered as a responsive Progressive Web Application. The architecture is organized into three layers:

- Client Layer — desktop, mobile, and tablet browsers, built with HTML, CSS, and JavaScript as a responsive PWA.
- Application Server Layer — a set of feature modules (see 3.2) implemented as client logic backed by Firebase services, communicating with the database over HTTPS.
- Database Layer — Cloud Firestore, holding users, marketplace listings, services/jobs/internships, messages/events, notifications/referrals, ratings/referrals, and announcements.

Firebase Authentication handles identity and institutional email verification; Cloud Firestore provides the primary NoSQL database with real-time synchronization; Cloud Storage holds images, portfolio files, and resumes; Cloud Functions automate server-side logic (referral tracking, rating aggregation, notification dispatch, malware scanning, and — per the payments addition in Section 6 — payment webhook handling); and Firebase Cloud Messaging (FCM) delivers push notifications.

3.2 System Modules

The system is organized into twelve modules (eleven original modules plus the Payments Module added to support the Digital Marketing team's monetization strategy):

Module	Responsibility
Authentication	Registration, secure login, email verification, profile management, institutional-email authentication, role management.
Marketplace	Create/edit/soft-delete listings, mark items sold, upload up to 5 images, view product details.
Services	Advertise skills, display portfolio links, set hourly rates, receive inquiries.
Internship & Jobs	Internship/job listings, resume upload, application submission, recruiter management.
Campus Events	Browse events, view details, track schedules and locations, RSVP.
Messaging (Chats)	Real-time messaging between buyers/sellers and clients/providers, with notifications and history.
Search & Filter	Keyword, category, price range, condition, and date filtering across all modules.
Notifications	In-app and email notifications, saved-search alerts, platform updates.
Referral	Generate referral links/codes, track referrals, maintain statistics.
Ratings & Reviews	Rate sellers and providers, leave written reviews, build trust.
Announcements	Admin-published campus announcements, notices, and platform updates.

Module	Responsibility
Payments (new)	Boosted-listing and premium-subscription payments via MTN Mobile Money / Orange Money, mediated by Cloud Functions.

3.3 Actors

- Student
- Verified Recruiter / Alumni
- Administrator (Campus Administrator / SGA)

3.4 Core Workflows

Four core workflows were documented as activity flows:

- Buying an Item — Login → Browse Marketplace → Search → Open Product → View Seller Rating → Contact Seller → Send Message → Seller Notified.
- Selling an Item — Login → Create Listing → Enter Product Details → Upload Images → Publish → Listing Appears → Buyer Notified → Receive Message → Complete Transaction.
- Finding an Internship — Login → Open Internship Portal → Search → View Details → Upload Resume → Submit Application.
- Referring a Student — Login → Generate Referral Code → Share Referral Link → Friend Registers → Referral Recorded.

3.5 Class Diagram (Entities)

The system's core entities, as modeled in the class diagram, are: User, Product (Marketplace listing), Service, Job, Event, Message, Notification, Referral, Rating, and Announcement — each with its own attributes and behaviors (e.g. Product: create(), update(), delete(), markSold()). Section 4 translates these entities into the Firestore-native schema actually implemented.

3.6 Technical Feasibility

The proposed architecture is confirmed technically feasible on the Firebase platform: Firebase Authentication with institutional email verification secures access; Cloud Firestore provides a scalable NoSQL store with real-time synchronization for listings, posts, jobs, events, notifications, referrals, ratings, and announcements; Cloud Storage secures images, portfolio files, and resumes with size/type validation; Cloud Firestore real-time listeners (onSnapshot) support immediate message delivery; FCM enables push notifications; Cloud Functions automate referral tracking, notification dispatch, rating aggregation, malware scanning, and announcement publishing; and Firestore queries with the Algolia extension provide fast search and filtering. All security requirements — HTTPS, Firestore Security Rules, authentication, role-based authorization, soft deletion, MIME-type validation, and malware scanning — are implementable on this stack without significant redesign.

4. Database Design (Cloud Firestore)

Prepared originally by Siela Lucien Noumboum (Team Member 5, Database Designer & Technical Documentation), this section translates the class diagram in Section 3.5 into a Firestore-native structure, since Firestore does not support relational joins across collections. Two techniques are used throughout: subcollections (nesting related data inside a parent document, e.g. messages inside a chat thread) and denormalization (duplicating small, frequently-needed fields, e.g. a seller's name and rating, directly onto related documents to avoid extra reads).

Naming note: this section uses the collection and field names confirmed by the live codebase, which is treated as authoritative over earlier document drafts — see Section 7 for the full reconciliation of naming differences across documents.

4.1 Collection Structure

- /users/{userId}
- /marketplace/{listingId}
- /services/{serviceId}
- /internships/{jobId} → /internships/{jobId}/applications/{applicationId}
- /events/{eventId} → /events/{eventId}/rsvps/{userId}
- /chats/{chatId} → /chats/{chatId}/messages/{messageId}
- /notifications/{notificationId}
- /referrals/{referralId}
- /ratings/{ratingId}
- /announcements/{announcementId}
- /transactions/{transactionId} (new — see Section 6)

4.2 Key Collections

users

Field	Type	Description
userId	string	Auto-generated by Firebase Authentication
name / email / role	string	Profile identity; role is student / recruiter / admin
profileImage	string	Cloud Storage URL
isVerified	boolean	Institutional email confirmed
avgRating / ratingCount	number	Denormalized, auto-updated by Cloud Function on new ratings
referralCode	string	Unique shareable code
isPremium / premiumUntil (new)	boolean / timestamp	Denormalized premium-subscription status

marketplace

Field	Type	Description
title / description / price / condition / category	mixed	Listing details, used by Search & Filter
status	string	available / sold / removed (soft-delete pattern)
imageURLs	array<string>	Up to 5 image URLs
sellerId / sellerName / sellerRating	mixed	Seller reference plus denormalized identity and rating
boostedUntil (new)	timestamp	Denormalized flag set by the payment webhook when a listing is boosted

chats

Field	Type	Description
participantIds	array<string>	Exactly two user IDs (buyer, seller/provider)
listingId	string	Optional — the listing/service the chat relates to
lastMessage / lastMessageAt	mixed	Denormalized preview and sort key for chat lists
messages (subcollection)	—	senderId, content, timestamp, status (sent/read)

4.3 Design Rationale

Decision	Reasoning
Messages nested under Chats	Enables efficient real-time listeners (onSnapshot) scoped to one thread, matching the Messaging Module's real-time requirement.
RSVPs nested under Events	Avoids unbounded array growth on the event document; supports efficient attendee counting.
Applications nested under Internships	Matches the natural access pattern — recruiters query applicants scoped to one posting.
Seller/provider identity denormalized onto listings/services	Avoids one extra read per card when rendering browse/search pages.
avgRating / ratingCount denormalized onto users	Avoids aggregating the full ratings collection on every profile view.
Soft-delete via status field	Matches the soft-delete requirement while avoiding a redundant parallel deleted_at field.

4.4 Security Rules — Structural Sketch

High-level Firestore Security Rules, aligned with the role-based authorization requirement:

```
match /marketplace/{listingId} {  allow read: if true;  allow create: if request.auth !=
null;  allow update, delete: if request.auth.uid == resource.data.sellerId  ||
getUserRole(request.auth.uid) == "admin"; } match /chats/{chatId} {  allow read, write: if
request.auth.uid in resource.data.participantIds; }
```

This is a planning-stage sketch; full rules require testing against the Firebase Emulator Suite before deployment.

5. UI/UX Design Documentation

5.1 User Flow

Home → Login/Register → Dashboard → choice of: Marketplace (Browse → Product Details → Purchase), Sell Item (Upload Product → Publish), Services (Book Service), Internships (View → Apply), Events (View → Register), or User Profile (Edit Profile) → Logout.

5.2 Information Architecture

Top-level navigation: Home, Login, Register, Dashboard, Marketplace (Categories, Search, Product Details, Checkout), Sell Item (Upload Images, Product Details, Publish), Services, Internships, Events, User Profile, Settings.

5.3 Key Screens

Screen	Key Elements
Home	Logo, search bar, menu, banner, categories, featured products, services/internships/events previews, footer.
Login / Register	Email, password (plus name, phone, confirm-password on registration); forgot-password and account links.
Marketplace	Search, categories, filters, product cards (image, name, price, action button).
Product Details	Image, name, price, description, seller, contact-seller and buy-now actions.
Sell Item	Image upload, name, category, price, description, publish action.
Services / Internships / Events	Search, cards (provider/company/venue), book/apply/register actions.
User Profile	Photo, name, email, phone, edit-profile, my listings, my purchases.
Dashboard	Welcome message, quick statistics, recent activity, notifications, shortcuts to all modules.
Settings	Account, privacy, notifications, security, language, theme, logout — plus a new Billing/Subscription section (Section 6).

6. Payment Gateway Addition

The Digital Marketing team's business strategy (Section 8) introduces a revenue model — boosted listings and premium provider accounts — that requires in-platform payment collection. The original Requirement Analysis document placed payment gateway processing out of scope; this has been formally revised, and a Payments Module has been added to the technical scope and implemented in the live codebase (paymentgateway.html, src/modules/paymentgateway/).

6.1 Chosen Provider

MTN Mobile Money and Orange Money were selected as the payment providers, reflecting the dominant mobile payment methods among students in Cameroon, and matching the FCFA pricing set by the Digital Marketing team.

6.2 Flow

- Client calls a callable Cloud Function (initiatePayment) with `userId`, `type` (boost / premium_subscription), and `amount`.
- The Cloud Function calls the MTN MoMo / Orange Money API server-side; provider credentials remain in Cloud Functions configuration and are never exposed to the client.
- The user completes payment via the provider's mobile money flow — CampusConnect never stores card or mobile money credentials directly.
- The provider sends a webhook on completion; an HTTPS-triggered Cloud Function verifies it and writes a record to `/transactions`.
- The same function updates a denormalized flag — `boostedUntil` on the listing, or `isPremium / premiumUntil` on the user — following the same pattern already used for `avgRating`.

6.3 Schema Addition

Field	Type	Description
<code>userId</code>	string	Reference to <code>users/{userId}</code>
<code>type</code>	string	boost premium_subscription
<code>amount / provider</code>	mixed	Payment amount and provider (mtn_momo / orange_money)
<code>gatewayRef</code>	string	Provider transaction reference — no payment credentials stored
<code>status</code>	string	pending completed failed
<code>createdAt</code>	timestamp	Transaction time

7. Design Consistency & Reconciliation

The project documents were developed by different team members working from a shared but evolving system model. As the design matured — and once the live codebase existed — small naming and one scope difference emerged across documents. This section records each difference and states the standard now in force, with the live codebase treated as authoritative wherever documents disagreed with it.

7.1 Collection & Module Naming

The Requirement Analysis document used `marketplace_items`, `chats`, `service_listings`, and `job_postings`. Siela's Database Design document proposed `listings`, `services`, `jobs`, and `conversations`. The live codebase settled on `marketplace`, `services`, `internships`, `events`, and `chats`.

Standard adopted

`marketplace`, `services`, `internships`, `events`, and `chats` are the standard collection/module names going forward — matching the live codebase. Siela's Database Design document should be revised to rename `listings` → `marketplace`, `jobs` → `internships`, and `conversations` → `chats`.

7.2 Messaging Thread Terminology

Resolved by 7.1: "chats" is the standing term (`chats.html`, `chats.js`, `/modules/chats/`), not "conversations." The codebase in this instance sided with the original SRS wording rather than the Database Design document's proposed rename.

7.3 Status Value Casing

The SRS and UC-01 used uppercase status values ("AVAILABLE", "SOLD"); the Database Design schema specified lowercase ("available", "sold", "removed"). Lowercase is recommended as the implementation standard, consistent with the schema and Firestore Security Rules; this should be confirmed against the actual field values written by `marketplace.js`.

7.4 Soft-Delete Implementation

The NFRD (NFR-SEC-02) and UC-01 specified a `deleted_at` timestamp field; the Database Design schema instead uses a status field with a "removed" value. The status-field pattern is adopted as the implementation standard, since it keeps deletion state consistent with the same field already used for `available/sold`, avoiding a redundant parallel field. This should be confirmed against the actual delete logic in `marketplace.js`.

7.5 Database Technology

Resolved

Confirmed and closed: the codebase is Firebase/Firestore only (src/config/firebase.js), with no relational database anywhere. Siela's Database Design document explicitly supersedes an earlier MySQL assumption; any remaining relational-database references elsewhere are outdated and should be removed.

7.6 Monetization vs. Payment Processing Scope

Resolved

The Digital Marketing team's revenue model required in-platform payment collection, which the original Requirement Analysis placed out of scope. This has been resolved by formally adding a Payments Module to the technical scope (Section 6), implemented using MTN Mobile Money and Orange Money and mediated entirely through Firebase Cloud Functions — consistent with the rest of the Firebase-only architecture.

7.7 Outstanding Items

Two points remain to be confirmed directly against the source code rather than the document set alone:

- The exact casing used for the status field values written by marketplace.js (Section 7.3).
- The exact soft-delete field/pattern implemented in marketplace.js's delete logic (Section 7.4).

8. Digital Marketing & E-Commerce Report

8.1 Brand Identity

CampusConnect's brand identity is built around Trust and Reliability — the brand needed to feel safe, dependable, and credible rather than flashy or casual, since the platform facilitates real transactions and exchanges between students.

Logo

Two interlocking "C" shapes: the left "C" incorporates a graduation cap (academics and student life); the right "C" incorporates connected people (community and collaboration) — together representing students connecting across campus life.

Color Palette

Color	Hex	Usage
Navy	#082B65	Primary — main headers and key brand elements
Indigo	#6024A2	Secondary — icons and supporting text
Purple	#752AB2	Accent — used sparingly for key actions (Sign Up, Sell Now)
White	#FFFFFF	Background and neutral spacing

Typography & Tone

- Headings: bold, clean sans-serif (Poppins or Montserrat). Body: Open Sans or Inter. No decorative or handwritten fonts.
- Tone of voice: clear and direct, warm but professional, always honest — no jargon or exaggeration.
- Imagery: real, natural-feeling photos or mockups of students, not staged stock photography.

Tagline

"Connect. Collaborate. Grow."

8.2 Market Research

Target Audience

University students who want to buy and sell items, offer services, find internship or job opportunities, and discover campus events.

Competitive Landscape

Competitor	Strengths	Weaknesses
Facebook Marketplace	Easy posting; built-in buyer chat	Not student-specific; no internships or campus events
OfferUp	Simple buy/sell app; trust ratings	Not student-focused; no services, internships, or events

Competitor	Strengths	Weaknesses
Campus WhatsApp/FB groups	Already used informally by students	No search/filter; disorganized; no dedicated sections

User Personas

- Sarah — The Seller: 2nd-year student selling textbooks/laptop; needs quick posting and trustworthy buyer communication.
- David — The Service Provider: 3rd-year student offering graphic design; needs a place to advertise services and showcase work.
- Amara — The Buyer/Browser: 1st-year student seeking affordable furniture, part-time work, and event info; needs strong search/filter and trustworthy listings.

Market Opportunities

- An all-in-one platform combining marketplace, services, internships, and events, unlike fragmented competitors.
- Built-in trust from being exclusive to verified university students.
- Search and filter functionality, a clear improvement over disorganized informal groups.
- Combined earning and career opportunities that no single competitor offers together.

8.3 Marketing Strategy

Go-to-Market

- Launch on one campus first to build trust before expanding.
- Recruit campus ambassadors to promote the platform and post initial listings.
- Target new students during orientation/welcome week, the peak demand period.
- Partner with student organizations, clubs, and hostel/dorm groups.
- Incentivize early listings so the platform doesn't launch empty.

Social Media

Primary platforms: Instagram (main channel), WhatsApp/Telegram (updates), TikTok (optional launch-week hype). Content pillars: how-it-works, student spotlights, countdown/hype, tips and reminders. Posting frequency: 2–3×/week during launch month, 1–2×/week afterward.

Launch Campaign

- Pre-launch (1 week before): "Coming Soon" teasers; ambassador outreach.
- Launch Day: official announcement across platforms and student groups; optional on-campus poster/QR presence.
- Launch Week Incentive: reward early adopters (e.g. first 50 sign-ups featured as top sellers).
- Follow-up (Week 2): share sign-up milestones and a student testimonial/spotlight.

Growth & Acquisition

- Referral system rewarding invites; campus-by-campus expansion of the launch playbook.
- Seasonal pushes aligned with semester start/end and internship season.

- Word of mouth as the primary channel, plus student union/club partnerships, orientation presence, and student ambassadors.

8.4 Content Creation

Landing page copy, a promotional flyer, product-launch graphics (countdown, feature highlight, live announcement, sign-up CTA), three launch social media posts, and a 35–40 second promotional video concept ("CampusConnect: One Platform, Endless Connections") were produced, all following the brand system in Section 8.1.

8.5 Business Strategy

Revenue Model

- Freemium listings — basic buy/sell/service listings remain free to drive adoption.
- Featured/boosted listings — small fee to appear at the top of search results or homepage.
- Premium accounts — optional monthly fee for service providers for extra visibility, a verified badge, and listing analytics.

Pricing

- Core marketplace and browsing: free for all students.
- Boosted listing: approx. 600–1,200 FCFA per boost.
- Premium provider account: approx. 1,800–3,000 FCFA/month.

(See Section 6 for how this monetization is now supported technically via MTN Mobile Money and Orange Money.)

Retention & Support

- Loyalty badges/featured placement for successful sales or referrals.
- Re-engagement notifications for saved-search matches; seasonal pushes at semester boundaries.
- Visible ratings/reviews to keep both sides accountable.
- In-app "Report an Issue" button, a monitored support channel (24-hour target response), an FAQ/Help Center, and an admin escalation path.

9. Status Summary & Next Steps

Workstream	Status
Requirement Analysis (SRS/FRD/NFRD)	Complete
System Architecture & UML	Complete
Database Design (Firestore)	Complete; naming reconciled against live code
UI/UX Design	Complete
Payment Gateway	Scoped and implemented (MTN MoMo / Orange Money); pending full end-to-end testing
Brand Development	Complete
Market Research	Complete
Marketing Strategy	Complete
Content Creation	Complete
Business Strategy	Complete; monetization now technically supported

Next Steps

- Confirm the two outstanding code-verification items in Section 7.7 (status casing, soft-delete pattern).
- Complete end-to-end testing of the payment flow (initiatePayment → provider checkout → webhook → Firestore update) against the Firebase Emulator Suite.
- Finalize Firestore Security Rules (Section 4.4) and test against the emulator suite before deployment.
- Full team review of all deliverables, followed by integration with the Software Engineering team's prototype ahead of the final presentation.